MPTA-Repair: Clock-Bound Repair for Multi-Property Timed Automata in Industrial Practice

***Abstract*—Timed automata (TAs) are widely used to model and verify real-time industrial systems, which are typically required to satisfy multiple predefined properties simultaneously. When verification fails, repairing the model is challenging because it requires modifying erroneous clock constraints to satisfy all properties while preserving functional equivalence. To address this, we present MPTA-Repair, an automated repair framework for multi-property TAs. The framework employs an iterative, counterexample-guided MaxSMT approach to generate minimally modified candidate models. To optimize search efficiency, it systematically exploits relationships among properties, counterexamples, and repair variables. Each candidate model is then validated through full-property regression verification and acceptability checks. Compared to the TARTAR baseline, MPTA-Repair achieves higher effectiveness on faulty models generated via our proposed mutation strategy. Specifically, it improves the success rate from 72% to 93% on benchmarks derived from UPPAAL and the XtaBenchmarkSuite, and from 20% to 76% on an industrial dataset constructed in this work.**

***Index Terms*—Timed automata, automated model repair, MaxSMT, multi-property verification**

I. Introduction

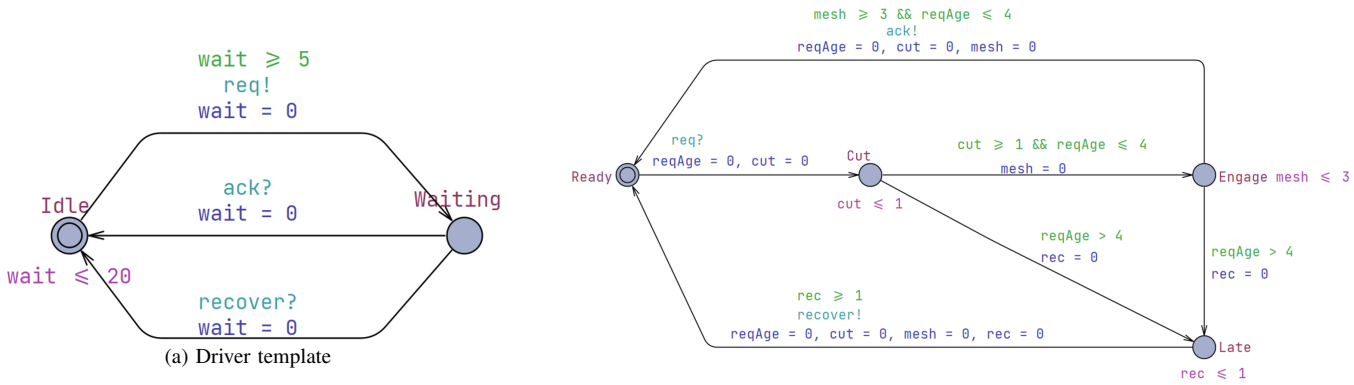
Timed automata (TAs) are widely used to model and verify real-time industrial systems whose correctness depends on both discrete behavior and quantitative timing constraints [1], [2]. Such systems commonly appear in railway control and communication protocols [3], automotive and autonomous-driving logic [4], [5], medical cyber-physical systems [6], [7], and aerospace or avionics scheduling [8]. In these domains, an alarm, pulse, message, or resource release may be functionally present but still incorrect if it occurs too early, too late, or for an incorrect duration. Verification therefore has to check not only reachability or event order, but also whether timing requirements are satisfied under all relevant executions.

Industrial timed-automata models are rarely validated against a single property. They are typically checked against a set of predefined properties that jointly express safety, bounded response, deadline satisfaction, error-state reachability, alarm duration, and deadlock freedom [9]. When a property is violated, model checkers such as UPPAAL, Kronos, or opaal can produce a timed diagnostic trace (TDT), namely a timed counterexample showing how the model reaches a violating state [10]. A TDT is useful evidence for repair, but it is not a repair instruction. It does not identify which clock constraint is erroneous, whether the error lies in a transition guard or a location invariant, or how a numerical bound should be changed to restore the intended timing behavior [11].

The difficulty becomes more pronounced in the multi-property setting. A timing fault may affect several properties, and different violated properties may produce related counterexamples. A repair that blocks one reported trace may leave another violating trace feasible, or may break a property that was previously satisfied. Similarly, changing a bound for one property can affect another property when the same clock, location, transition, synchronization, or execution fragment is involved. Thus, a practical repair method for industrial timed automata must not accept a candidate merely because it fixes one local counterexample. It must validate the candidate against the complete property set and ensure that the functional behavior of the model is preserved.

Figure 1 illustrates this issue with a simplified network timed automaton for an automotive gear-shift controller [4]. A Driver template periodically issues a shift request, and a GearController template performs torque cut, gear engagement, acknowledgement, and recovery. The model uses $cut \leq 1$ in the Cut location, $cut \geq 1$ on the transition to Engage, $mesh \leq 3$ in the Engage location, and $mesh \geq 3$ on the acknowledgement transition. One property requires at least two time units of torque cut before gear engagement, while another property requires the shift request to be acknowledged within four time units. A single-property repair for the first property may change only the cut bounds from 1 to 2 and thereby block the early-engagement trace. However, with the mesh bounds unchanged at 3, the earliest acknowledgement would occur at time 5, so the response-deadline property fails. A multi-property repair instead changes both the cut bounds and the mesh bounds, for example to $cut \leq 2$, $cut \geq 2$, $mesh \leq 2$, and $mesh \geq 2$. The repaired controller then satisfies both the minimum torque-cut requirement and the response deadline while preserving the same untimed request-acknowledgement behavior. This example motivates treating properties, counterexamples, and repair variables as joint repair evidence rather than independent local repair tasks.

Existing repair techniques provide an important foundation for this problem. Clock-bound repair encodes a TDT as linear real arithmetic, introduces variation variables for clock bounds, and uses MaxSMT solving to compute small modifications to guards and invariants [11]. TARTAR builds on this idea by suggesting syntactic repairs and checking whether the repaired model preserves the untimed functional behavior of the original network timed automaton [12]. Related work also includes parameter synthesis for parametric timed automata



(a) Driver template

(b) GearController template

Fig. 1. A simplified network timed automaton for a cyclic automotive gear-shift controller. The faulty model uses $cut \leq 1$, $cut \geq 1$, $mesh \leq 3$, and $mesh \geq 3$. A repair that changes only the cut bounds blocks the early-engagement trace but can violate the response-deadline property; a multi-property repair also changes the mesh bounds.

[13], clock-guard repair through abstraction and testing [14], robustness analysis for uncertain timing constants [15], and SAT/SMT-based model or program repair [16]. These methods show that automated repair of timed models is feasible, but most TDT-based workflows still reason from one violated property or one diagnostic trace at a time.

This paper presents **MPTA-Repair**, an automated repair framework for multi-property TAs. Given a faulty timed-automata model, a set of timing properties, and diagnostic traces produced by verification, MPTA-Repair searches for small changes to numerical clock bounds in transition guards and location invariants. It does not change property formulas, locations, transitions, synchronization labels, clock references, or resets. This restriction matches the clock-bound repair formulation and the mutation-based evaluation used in this paper, while broader modeling faults such as reset errors, synchronization errors, data-update errors, or structural errors are outside the current scope.

MPTA-Repair employs an iterative, counterexample-guided MaxSMT approach to generate minimally modified candidate models. Instead of treating violated properties and diagnostic traces as independent subproblems, it maintains a pool of property-trace repair obligations and searches for a clock-bound assignment that satisfies the currently encoded obligations. To improve search efficiency, the framework systematically exploits relationships among properties, counterexamples, and repair variables to organize the search. These relations group related evidence, rank or restrict active repair variables, and avoid solving over unrelated bounds when the current counterexample evidence gives no reason to include them. These relations guide candidate generation, whereas accepted candidates must still satisfy all properties in regression verification and pass a TARTAR-style acceptability check based on functional equivalence.

We evaluate MPTA-Repair on faulty models generated via a mutation strategy designed for clock-bound repair. The benchmark dataset is derived from UPPAAL and the XtaBenchmarkSuite, while the industrial dataset is constructed in this work from practical case studies covering railway alarm and communication scenarios [3], [17], train-gate behavior [18], automotive gear control [4], cardiac pacing [6], autonomous road-junction rules [5], mechanical ventilation [7], and aerospace scheduling [8]. We measure repair effectiveness, repair size, runtime, regression outcomes, and behavior preservation. Compared with an iterative TARTAR-style baseline under the same repair space, full-property regression requirement, and admissibility requirement, MPTA-Repair improves the repair success rate from 72% to 93% on the benchmark dataset and from 20% to 76% on the industrial dataset.

The contributions of this paper are as follows. First, we formulate clock-bound repair for practical timed-automata models as a multi-property, multi-counterexample repair problem in which local trace repair is insufficient unless the repaired model satisfies the complete property set. Second, we present MPTA-Repair, an iterative counterexample-guided MaxSMT framework that uses relationships among properties, counterexamples, and repair variables to guide the repair search. Third, we implement a prototype that combines trace analysis, MaxSMT-based clock-bound repair, full-property regression verification, and TARTAR-style admissibility checking. Fourth, we evaluate the approach on benchmark and industrial datasets against an iterative single-property baseline under identical acceptance criteria. Finally, we report practical lessons on property selection, repair interpretability, counterexample accumulation, and the need for full regression after each candidate repair.

The remainder of this paper is organized as follows. Section II provides the background and problem setting, including timed automata, diagnostic traces, clock-bound repair, and the multi-property repair objective. Section III presents MPTA-Repair and its relation-guided repair workflow. Section IV presents the implementation and experimental evaluation, including datasets, results, and practical lessons. Section V concludes the paper.

II. Background and Motivation

A. Timed Automata and Industrial Networks

Timed automata provide a standard formal model for systems whose correctness depends on both discrete control decisions and quantitative time constraints [1], [2]. Let C be a finite set of clocks. A clock constraint over C is a conjunction of atomic constraints of the form $x \sim b$, where $x \in C$, $\sim \in \{<, \leq, =, \geq, >\}$, and b is a non-negative numerical bound. We write $\mathcal{B}(C)$ for the set of such constraints. A timed automaton is a tuple

$$T = (L, \ell_0, C, \Sigma, \Theta, I),$$

where L is a finite set of locations, $\ell_0 \in L$ is the initial location, Σ is a set of action labels, Θ is a finite set of transitions, and $I : L \to \mathcal{B}(C)$ assigns an invariant to each location. A transition $\theta \in \Theta$ is written as

$$\theta = (\ell, g, a, R, \ell'),$$

where ℓ and ℓ' are the source and target locations, $g \in \mathcal{B}(C)$ is the guard, $a \in \Sigma$ is the action label, and $R \subseteq C$ is the set of clocks reset by the transition.

The semantics is defined over states (ℓ, u), where ℓ is a location and $u : C \to \mathbb{R}_{\geq 0}$ is a clock valuation. A delay step lets time elapse in the current location as long as the location invariant remains satisfied. A discrete step can be taken when the current clock valuation satisfies the transition guard; the step moves the automaton to the target location and resets the clocks in R. Thus, guards constrain when discrete actions are enabled, whereas invariants constrain how long the automaton may remain in a location.

Industrial timed-automata models are usually networks rather than single automata. A network

$$\mathcal{N} = T_1 \parallel \cdots \parallel T_k$$

composes templates for controllers, plants, environments, communication protocols, schedulers, and monitors [19]. The exact product semantics depends on the modeling language and tool, for example on binary synchronization, broadcast channels, urgent or committed locations, and data variables. In this work these structural and data-level features are treated as fixed modeling context. The repair space is deliberately restricted to numerical clock-bound occurrences in guards and invariants; synchronization labels, reset sets, clock references, locations, transitions, data updates, and property formulas are not repair variables.

This restriction matches a common class of industrial modeling faults. Timing constants often represent design thresholds, sampling periods, timeout values, alarm durations, refractory intervals, or scheduling deadlines. For example, vehicle controllers may need to complete a gear-shifting sequence within a bounded response time [4]; railway models may encode gate, alarm, or communication timeout requirements [3]; medical devices may encode physiological timing windows [6]; and avionics or aerospace models may encode execution-time bounds and deadlines [20]. In such models, an incorrect numerical bound can make an otherwise intended behavior occur too early, too late, or for an invalid duration.

We do not assume that all modeling errors are clock-bound errors. Industrial models may also contain wrong resets, missing transitions, incorrect synchronizations, erroneous data updates, or incomplete properties. These faults are outside the current repair space. The assumption in this paper is narrower: the discrete model structure is treated as the intended design, while one or more numerical timing bounds may be inaccurate due to calibration, transcription, dimensioning, or design-space uncertainty. This is the fault model targeted by clock-bound repair [11] and is consistent with mutation-based repair and testing, where faults are modeled as small deviations from an otherwise near-correct artifact [21].

B. Properties and Timed Diagnostic Traces

Timed-automata models are checked against timing properties using real-time model checkers such as UPPAAL [19]. This paper focuses on safety-style verification obligations. Direct examples include queries of the form $A[]\,\psi$, where every reachable state must satisfy the state predicate ψ, and deadlock-freedom checks such as $A[]\ not\ deadlock$. Bounded-response and deadline requirements are handled when they are encoded by observer or monitor templates into safety-style violations, for example by reaching an error or late location [9]. Therefore, the repair procedure reasons about property violations that can be represented as reachable safety violations in the instrumented timed-automata network.

A discrete trace skeleton is a finite sequence of transitions

$$\tau = \theta_0, \theta_1, \ldots, \theta_{n-1}.$$

A timed realization of τ is a sequence of non-negative delays and clock valuations that makes the transition sequence executable from the initial state while respecting guards, resets, invariants, and network synchronization. The skeleton τ is feasible if it has at least one timed realization. Given a property φ, a feasible skeleton is a timed diagnostic trace (TDT) for φ if at least one of its timed realizations reaches a state that violates the safety condition associated with φ [11]. Depending on the model checker, the reported counterexample may contain concrete delays, symbolic zones, or only enough information to reconstruct the discrete path and timing constraints.

A TDT is evidence of failure, not a root-cause explanation. It shows that some timing realization of some discrete behavior violates a property, but it does not identify which numerical bound is wrong, whether the fault lies in a guard or invariant, or how the model should be changed. Several bounds may jointly enable the violating realization, and one incorrect bound may contribute to several diagnostic traces. Conversely, blocking the discrete path of a TDT is usually not a valid repair: it may remove intended functional behavior, such as a request, acknowledgement, actuation, communication step, or scheduling decision. A useful repair should change the timing envelope of the behavior while preserving the relevant untimed functionality of the model.

C. Clock-Bound Repair and Admissibility

A repairable site is an occurrence of a numerical bound b in an atomic clock constraint $x \sim b$ that appears in a transition guard or a location invariant. A clock-bound repair changes this occurrence to a new value b', equivalently by introducing a variation variable v and using the repaired bound $b + v$. The comparison operator, clock reference, transition structure, location structure, synchronization label, and reset set are kept fixed. Bounds are constrained to remain in the numerical domain supported by the target model representation.

TDT-based clock-bound repair encodes the feasibility of a diagnostic trace as constraints over delays and clock values, then introduces variation variables for the bounds that occur in the encoded guards and invariants [11]. MaxSMT solving can then be used to prefer small repairs, for example by maximizing soft constraints of the form $v = 0$ and, when needed, minimizing the magnitude of nonzero variations [22]. In the single-trace setting, the resulting candidate is local: the same discrete trace skeleton should remain feasible, but no feasible realization of that skeleton should still witness the considered property violation.

Local trace repair is not sufficient for model repair. A bound change that eliminates one violating realization may create a new violation on another execution, fail another property, or make intended functionality unavailable. MPTA-Repair therefore treats MaxSMT-based trace repair as candidate generation only. A candidate is accepted only after two global checks. First, the repaired model must satisfy the complete property set under regression verification. Second, it must pass an admissibility check based on TARTAR-style functional equivalence [11].

The admissibility check compares untimed functional behavior rather than timed language equality. This distinction is necessary because a successful timing repair is expected to remove some violating timed realizations. Let Σ_f denote the functional alphabet used for admissibility, after excluding monitor-only, observer-only, and internal labels when appropriate. The intended criterion is

$$\mathrm{Untimed}_{\Sigma_f}(M_{bad}) = \mathrm{Untimed}_{\Sigma_f}(M_{rep}),$$

meaning that the repaired model should not add or remove functional action traces, although the timing of those traces may change. This criterion prevents repairs that satisfy safety properties merely by disabling intended behavior.

D. Why Multi-Property Repair Is Needed

Industrial timed-automata models are normally validated by a set of properties rather than by a single assertion [9]. The properties jointly describe the reliability envelope of the model, including absence of unsafe states, bounded response, timeout handling, deadline satisfaction, admissible residence times, and deadlock freedom. Let M_{bad} be a faulty timed-automata network and let

$$\Phi = \{\varphi_1, \ldots, \varphi_m\}$$

be the property set used to validate it. The goal is to construct a repaired model M_{rep} by changing a small number of clock-bound constants in guards and invariants such that:

$$M_{rep} \models \Phi,$$

the repaired bounds are well formed, and the repaired model preserves the projected untimed functional behavior of M_{bad}.

The need for joint reasoning can be seen in the gear-shift controller used as a running example. The driver periodically issues a shift request, and the controller performs torque cut, gear engagement, acknowledgement, and recovery. Suppose the faulty controller uses bounds such as $cut \leq 1$, $cut \geq 1$, $mesh \leq 3$, and $mesh \geq 3$. One property may require at least two time units of torque cut before gear engagement, while another property may require acknowledgement within four time units of the request. A single-property repair that changes only the cut bounds from 1 to 2 can block the early-engagement violation. However, if the $mesh$ bounds remain at 3, the earliest acknowledgement may occur at time 5, thereby violating the response deadline. A multi-property repair must instead consider both requirements and may need to change both the cut and $mesh$ timing bounds while preserving the same untimed request–engage–acknowledge behavior.

This example illustrates why violated properties, diagnostic traces, and repair variables should not be treated as unrelated subproblems. Different properties may share clocks, locations, synchronizations, or execution fragments. Multiple TDTs may expose different realizations of the same underlying timing defect. Conversely, a repair computed from one trace can be locally valid yet globally unacceptable after full regression. MPTA-Repair addresses this setting by accumulating multi-property and multi-counterexample repair evidence, generating small clock-bound candidates with MaxSMT, and accepting a candidate only after complete property regression and functional-admissibility checking.

III. Approach

A. Overview

MPTA-Repair is a counterexample-guided framework designed for the multi-property, multi-counterexample clock-bound repair of industrial timed-automata models [23]. The framework takes as input a faulty model M, a property set $\Phi = \{\varphi_1, \ldots, \varphi_m\}$, and configuration parameters including the maximum number of refinement rounds, candidate limits, and a global timeout. Rather than requiring separate user input, repairable bounds are directly extracted from the numerical clock bounds within transition guards and location invariants [11]. The output is either a repaired model M' along with a set of clock-bound edits, or a failure report indicating that no admissible full-property repair was identified within the allocated budget.

The core architecture operates on two primary insights. While candidate repair generation can be driven by local diagnostic traces, candidate validation must remain global. A candidate that resolves a single violated property might induce failures in others, because shared clocks, locations,

transitions, or synchronizations often participate in multiple timing requirements. Consequently, MPTA-Repair iteratively collects counterexamples from newly violated properties to progressively constrain the admissible repair candidates. This refinement loop continues until a candidate satisfies the entire property set and passes admissibility verification, or until the configured budget is exhausted. For instance, a fixed timeout of 10 minutes per repair instance can be enforced to prevent unbounded refinement [24].

Multiple properties and diagnostic traces are treated as interconnected elements rather than isolated subproblems. A single erroneous timing bound can contribute to multiple violations, and distinct traces may manifest different executions of the same underlying fault. To leverage this interdependence, MPTA-Repair maintains a joint pool of property-trace obligations and searches for a clock-bound assignment that simultaneously satisfies all encoded constraints. Relation analysis across properties, traces, and repairable bounds minimizes redundant computation and guides search-space exploration, while formal verification remains the final correctness boundary [23].

In operation, MPTA-Repair first verifies the model against the property set, collects timed diagnostic traces (TDTs) for any violations, extracts repairable clock bounds, and constructs a joint MaxSMT repair problem from the active property-trace pool. The computed assignment is then instantiated into a candidate model, which is validated against the complete property set and a TARTAR-style admissibility criterion [11]. If validation fails, the assignment is blocked, newly identified diagnostic traces are integrated into the pool, and the refinement loop repeats.

Figure 2 illustrates the overall workflow. The initial verification stage populates the property-trace pool, the relation-guided optimization phase structures dependencies among properties, counterexamples, and repairable bounds, and the MaxSMT solving stage generates candidate edits. Finally, the validation component either accepts a fully admissible repair or feeds new diagnostic evidence back into the subsequent refinement round.

B. Joint Trace Encoding and Optimization Objective

MPTA-Repair adopts the bound-variation paradigm for clock-bound repair [11]. Each modifiable numerical bound in a transition guard or location invariant is associated with a variation variable, ensuring that the repaired constraint retains the original clock reference and comparison operator while incorporating a shifted numerical value. Consequently, the repair space is restricted to clock-bound constants, whereas transition structures, location topologies, synchronization labels, reset sets, and discrete updates remain unaltered.

For each diagnostic trace, the framework constructs a TARTAR-style trace constraint system [11] encompassing standard conditions for clock initialization, time elapse, clock resets, guard satisfaction, and invariant satisfaction. By replacing original clock bounds with their varied counterparts, this formulation preserves the discrete trace skeleton within the solver while allowing its timing region to change. For a violated property and diagnostic trace, the resulting obligation serves two purposes. It ensures post-repair executability of the discrete trace skeleton to prevent trivial elimination of functional paths, and it requires that no feasible timing realization of this skeleton violates the target property. Local obligations therefore isolate violating timed behaviors without blocking the underlying functional execution.

To guarantee physical validity during candidate generation, static consistency constraints are embedded directly into the formulation [25]. Repaired bounds must lie within the supported numerical domain. Furthermore, when a lower bound and an upper bound jointly constrain the same clock within a local region, their repaired values must maintain a consistent timing interval.

From the active pool of property-trace obligations, MPTA-Repair assembles a joint MaxSMT problem. The hard constraints include all encoded trace obligations, static consistency requirements, well-formedness conditions for repaired bounds, and blocking constraints derived from previously rejected candidates. The optimization objective is governed by a hierarchical set of simple soft constraints [22]. The primary objective minimizes the number of altered clock bounds by maximizing soft constraints of the form $v = 0$. Among candidate solutions with the same number of modifications, a secondary objective minimizes the total magnitude of the variations. This optimization strategy aligns with the engineering goal of generating minimal, human-inspectable repairs and deliberately avoids speculative metrics such as semantic risk or causal relevance. Relation analysis is instead used externally to select or rank repair variables and obligations before and around the solving phase.

C. Relation-Guided Search Optimization

During successive refinement rounds, the accumulation of violated properties, diagnostic traces, and editable clock bounds can significantly expand the search space [23]. Unstructured encoding of all property-trace obligations and repairable bounds increases the complexity of the MaxSMT problem and can induce redundant solving cycles over overlapping evidence. To mitigate this overhead, MPTA-Repair introduces relation-guided optimization to prune, group, and rank variables and constraints. These relation-based heuristics serve strictly as search optimizations; validity and admissibility of generated candidates are still guaranteed through full-property regression and formal verification.

At the property layer, MPTA-Repair operates on safety properties normalized into an `A[]` fragment over location predicates and clock or integer constraints [9]. Relation analysis uses conservative syntactic and structural rules to evaluate equivalence and dominance. Two properties are equivalent only when their normalized representations are syntactically identical. Dominance is established only when clear logical subsumption holds, for example when two upper-bound properties share identical structures but differ in their numerical thresholds and one threshold strictly implies the other. Shared

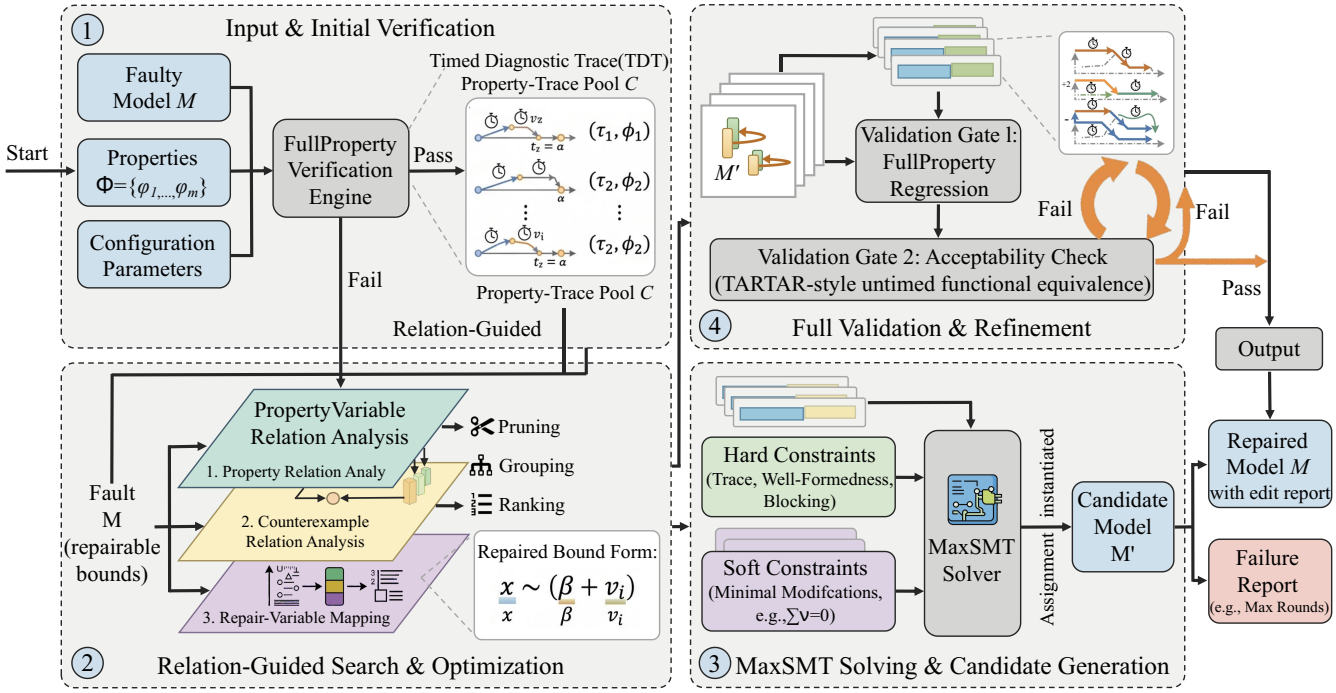


Fig. 2. Overview of the MPTA-Repair workflow.

clocks, target locations, and template structures are tracked as dependency metadata, but they do not independently justify removing obligations.

At the counterexample layer, the framework maintains a managed pool of TDTs indexed by their corresponding violated properties. Redundant traces are eliminated through exact-duplicate removal. The remaining TDTs are annotated with structural features, including traversed transitions, visited locations, active clocks, guard and invariant bounds, and originating properties [11]. Structural similarity, common prefixes, and shared repair variables are used to cluster related traces and orchestrate joint solving. A trace obligation is deactivated only when an exact duplicate is detected or when a sound subsumption check proves that its local constraint profile is covered by an existing encoded obligation.

At the repair-bound layer, MPTA-Repair establishes a mapping between each active property-trace obligation and the clock bounds capable of influencing its feasibility. This mapping restricts the active decision variables in the MaxSMT problem to those directly relevant to the current counterexample pool, while providing a heuristic ordering for candidate generation. Rather than imposing rigid structural constraints or hard-coding specific modifications, relation scores serve as an adaptive guidance mechanism. They improve search efficiency by dynamically deactivating irrelevant repair variables, ensuring that the solver concentrates on dimensions supported by counterexample evidence.

D. Validation, Admissibility, and Refinement

Candidate validation serves as the primary correctness guarantee in MPTA-Repair [26]. After a candidate edit set is generated, the repaired model is verified against the entire property set Φ. If a candidate resolves the triggering violation but introduces a failure in another property, it is rejected, indicating that the current property-trace pool is under-constrained. MPTA-Repair then extracts the newly observed diagnostic trace and appends it to the pool, steering the subsequent refinement round.

Candidates that pass full-property regression are further evaluated against a TARTAR-style admissibility criterion based on functional equivalence [11]. This step ensures that clock-bound modifications do not satisfy safety properties by disabling intended system behaviors. Rather than enforcing timed-language equivalence, which would conflict with the goal of eliminating violating timed paths, admissibility is checked by constructing untimed automata from both the original and repaired models and comparing their accepted untimed languages [27]. A candidate is rejected if it adds or removes discrete functional paths, ensuring that only timing characteristics are changed.

The refinement loop therefore establishes a closed-loop mechanism that pairs local candidate generation with global validation. Local MaxSMT formulations propose candidate assignments, full-property regression ensures complete specification compliance, and functional-equivalence checking preserves the underlying discrete structure. Blocking constraints

prevent recurrence of failed assignments, ultimately yielding a repair that satisfies the multi-property specification while preserving the intended untimed model semantics.

IV. Implementation and Experimental Evaluation

This section presents the implementation of MPTA-Repair and evaluates its performance on both benchmark and industrial case studies. To ensure practical utility, our approach focuses exclusively on repairing numerical clock bounds within transition guards and location invariants, rather than altering the discrete transition logic. This parametric repair space is suitable for industrial systems, as it corrects clerical errors and optimizes timing resources without disrupting the underlying functional design. In addition, comparing MPTA-Repair with an iterative baseline extended from the TARTAR paradigm [12] demonstrates that multi-property regression and multi-counterexample analysis are essential to prevent secondary errors.

A. Prototype Implementation

We implemented MPTA-Repair as an incremental pipeline that takes a faulty model and produces a validated repaired model or a failure report. The tool integrates a timed-automata parser, a verification interface using UPPAAL [19], a Z3-powered MaxSMT backend [25], and an admissibility checker. Modeled after a control-loop architecture, the pipeline executes the following steps.

Input and initial verification. The pipeline reads the faulty model, properties, and configurations. The verification engine extracts editable clock-bound variables from transition guards and location invariants, generating an initial trace pool of timed diagnostic traces for all violated properties.

Relation-guided search and optimization. This module maps the diagnostic traces to the extracted variables. A relation-analysis mechanism prunes, groups, and ranks these variables by identifying logical dependencies among properties, counterexamples, and repair locations. This mapping structures the repair constraints and reduces the search space.

MaxSMT solving and candidate generation. The optimized data is passed to the MaxSMT solver, which constructs a constraint system. This system includes hard constraints for trace feasibility, well-formedness, and blocking criteria, along with soft constraints designed to minimize modifications. Once a solution is found, a model writer generates the candidate model.

Full validation and refinement. The candidate model must pass two validation steps to ensure timing correctness and behavioral equivalence. First, full-property regression verification through UPPAAL ensures that no new violations are introduced. Second, an acceptability check evaluates untimed functional equivalence using a TARTAR-style criterion [11], rejecting candidates that alter the functional, untimed behavior.

Iteration. If a candidate fails either validation step, the control loop feeds the newly discovered diagnostic traces back into the trace pool. This initiates a counterexample-guided refinement loop, updating the solver constraints until a globally valid repair is found or the search budget is exhausted.

TABLE I
Statistics of the benchmark and industrial datasets.

Dataset	Original models	Mutated models	Avg. properties/model
Benchmark	9	54	4.1
Industrial	4	105	7.3

B. Evaluation Strategy and Datasets

To evaluate MPTA-Repair, we use two datasets with different characteristics, summarized in Table I. The benchmark dataset includes nine standard models from UPPAAL and the XtaBenchmarkSuite, such as Elevator and FDDI, providing structural diversity for baseline comparison. The industrial dataset contains four closely coupled industrial case studies: Gear Control System for automotive transmission, SAE RoR for autonomous-driving decisions, Train Controller Alarm for railway emergency timing, and Train Gate Controller for shared-resource concurrency. Unlike the benchmarks, these industrial models contain dense timing constraints across multiple properties.

Since real-world faulty timed automata are rare, we synthesize faulty instances using mutation [24]. Fault injection modifies only numerical clock-bound constants in guards and invariants, excluding structural changes. Simple mutants contain a single localized modification, while complex mutants combine two non-redundant single-point faults. We filter these combinations to ensure that both faults contribute to the property violation. All retained mutants are syntactically valid, violate at least one timing safety specification, and pass an initial admissibility check to ensure their original untimed functional behavior is preserved before repair.

We compare MPTA-Repair against an adapted iterative TARTAR-style baseline [12]. For fairness, this baseline uses the same regression and admissibility validation steps as our method. The primary difference is that the baseline derives repairs from only one property or diagnostic trace at a time. Because it lacks the joint multi-counterexample relation analysis used in our framework, the baseline often gets stuck in local optima and overfits to isolated fault paths. This comparison highlights the importance of analyzing properties, counterexamples, and repair variables simultaneously.

C. Experimental Results

The experimental evaluation demonstrates that MPTA-Repair outperforms the iterative baseline, particularly on complex industrial models. As shown in Table II, the baseline achieves a 72% success rate on the benchmark dataset, whereas MPTA-Repair reaches 93%. This performance gap widens on the industrial dataset, where the baseline repairs only 20% of the cases compared to 76% for MPTA-Repair. These results indicate that our approach remains effective on standard timed-automata benchmarks while demonstrating robustness in complex industrial systems where properties and traces interact closely.

The baseline is competitive on simpler benchmark models because timing errors are frequently concentrated on isolated

TABLE II
REPAIR SUCCESS RATES ACROSS BENCHMARK AND INDUSTRIAL DATASETS.

Dataset	Method	Success rate
Benchmark	Iterative baseline	72%
Benchmark	MPTA-Repair	93%
Industrial	Iterative baseline	20%
Industrial	MPTA-Repair	76%

guards or invariants. In these localized scenarios, a single diagnostic trace often provides sufficient information to infer a correct repair. However, even within standard benchmarks featuring shared multi-channel structures, such as the FDDI protocol, MPTA-Repair is more robust because it prevents single-trace overfitting. This advantage becomes more pronounced in complex industrial models that feature interacting concurrent components, urgent or broadcast synchronization channels, observer templates, and array-based constructs. In these settings, localized repairs by the baseline frequently fail subsequent global validation checks. Our empirical analysis shows that the baseline lacks explicit optimization to minimize edit magnitude, often generating extreme candidate bounds that violate admissibility. Additionally, the baseline struggles with regression verification over three to nine coupled properties because it does not analyze their corresponding counterexamples jointly.

MPTA-Repair achieves higher success rates due to two primary design advantages. First, multi-counterexample accumulation reduces overfitting to a single timed diagnostic trace, which typically exposes only one timing realization of an underlying fault. By accumulating and analyzing diagnostic traces from multiple operational modes, the framework synthesizes minimal joint edit sets that address the timing fault comprehensively. Second, relation-guided optimization complements precise MaxSMT objectives to improve candidate quality. By identifying diagnostic traces that share path fragments or target repair variables, the solver focuses on modifying numerical clock bounds close to the observed violations while penalizing large edit magnitudes. Successful repairs therefore typically require modifying only one or two critical clock bounds, which facilitates subsequent manual inspection and verification.

D. Discussion and Practical Lessons

The evaluation highlights several insights for applying automated repair to industrial timed automata. First, full-property regression testing is necessary. Because industrial system properties are interdependent, a localized fix designed solely to satisfy a liveness-like response monitor can inadvertently violate a separate safety monitor that constrains maximum waiting times. MPTA-Repair avoids these behavioral regressions by treating overall property restoration as a global objective. Second, admissibility is critical to measuring repair success. Without ensuring structural and functional equivalence, a solver might satisfy a specific timing property by disabling the intended functional behavior, such as preventing a critical system action entirely. The strict admissibility check ensures that property satisfaction is not achieved at the expense of untimed functional behavior.

Our failure analysis reveals several limitations of the current repair space. Computational timeouts and unsupported model syntax account for only a portion of the unsuccessful repair attempts. In tightly coupled domains such as cardiac pacing, certain complex errors require simultaneous, consistent adjustments across both guards and invariants, which significantly expands the search space. Additionally, incomplete diagnostic trace coverage from the verification engine can occasionally lead to overfitting. Certain complex cases lacked any feasible candidate within the restricted clock-bound parameter space. This limitation suggests that repairing these models requires broader structural modifications, such as altering synchronization labels, adjusting clock resets, or modifying discrete data updates.

V. CONCLUSION

This paper presented **MPTA-Repair**, an automated workflow for multi-property and multi-counterexample clock-bound repair of industrial timed automata. To overcome the limitations of isolated single-trace fixes, the method accumulates diagnostic traces and exploits explicit relations among properties, counterexamples, and repairable bounds to guide a MaxSMT-based search. Each synthesized candidate is validated through full-property regression and an admissibility check based on untimed functional equivalence [11]. This design ensures that accepted repairs are system-level timing corrections that preserve the intended operational behavior.

Evaluations across benchmark and industrial datasets show that MPTA-Repair outperforms the iterative baseline, particularly in industrial systems with dense property interactions. The failure analysis further indicates that unsuccessful repairs often stem from the intrinsic limits of a strictly parametric clock-bound repair space.

Future work will primarily focus on extending the automated repair space beyond numerical clock bounds to include broader structural modifications, such as altering synchronization labels and redesigning discrete control logic. We also plan to investigate the theoretical and algorithmic relationship between candidate repair generation and admissibility verification. By formally integrating behavioral admissibility constraints into the early stages of search-space formulation, we aim to proactively prune functionally invalid candidates and thereby improve computational efficiency for large-scale industrial applications.

REFERENCES

[1] R. Alur and D. L. Dill, "A theory of timed automata," *Theoretical Computer Science*, vol. 126, no. 2, pp. 183–235, 1994.

[2] J. Bengtsson and W. Yi, "Timed automata: Semantics, algorithms and tools," in *Lectures on Concurrency and Petri Nets*, ser. Lecture Notes in Computer Science. Springer, 2004, vol. 3098, pp. 87–124.

[3] D. Basile, A. Fantechi, and I. Rosadi, "Formal analysis of the UNISIG safety application intermediate sub-layer," in *Proceedings of Formal Methods for Industrial Critical Systems*, 2021, pp. 176–193.

[4] M. Lindahl, P. Pettersson, and W. Yi, "Formal design and analysis of a gear controller," in *Proceedings of Tools and Algorithms for the Construction and Analysis of Systems*, 1998, pp. 281–297.

[5] A. Belguidoum, A. S. Al-Sibahi *et al.*, "A double-level model checking approach for an agent-based autonomous vehicle and road junction regulations," *Electronics*, vol. 10, no. 23, p. 3040, 2021.

[6] Z. Jiang, M. Pajic, R. Alur, and R. Mangharam, "Modeling and verification of a dual chamber implantable pacemaker," in *Proceedings of the IEEE Real-Time and Embedded Technology and Applications Symposium*, 2012, pp. 188–203.

[7] J. Cuartas *et al.*, "Formal verification of a mechanical ventilator using UPPAAL," in *Proceedings of Formal Techniques for Safety-Critical Systems*, 2023.

[8] A. David, K. G. Larsen, A. Legay, M. Mikučionis, and D. B. Poulsen, "Herschel-planck software schedulability analysis using UPPAAL," in *Proceedings of Leveraging Applications of Formal Methods, Verification and Validation*, 2010, pp. 282–296.

[9] T. Vogel, M. Carwehl, G. N. Rodrigues, and L. Grunske, "A property specification pattern catalog for real-time system verification with UPPAAL," 2022, arXiv:2211.03817.

[10] C. Daws, A. Olivero, S. Tripakis, and S. Yovine, "The tool KRONOS," in *Hybrid Systems III*, ser. Lecture Notes in Computer Science. Springer, 1996, vol. 1066, pp. 208–219.

[11] M. Koelbl, S. Leue, and T. Wies, "Clock bound repair for timed systems," 2019, manuscript.

[12] ——, "TarTar: A timed automata repair tool," 2020, arXiv:2002.02760.

[13] R. Alur, T. A. Henzinger, and M. Y. Vardi, "Parametric real-time reasoning," in *Proceedings of the ACM Symposium on Theory of Computing*, 1993, pp. 592–601.

[14] Étienne André, P. Arcaini, A. Gargantini, and M. Radavelli, "Repairing timed automata clock guards through abstraction and testing," in *Proceedings of Tests and Proofs*, 2019.

[15] J. Bendík, A. Sencan, E. A. Gol, and I. Černá, "Timed automata robustness analysis via model checking," 2021, arXiv:2108.08018.

[16] M. Jose and R. Majumdar, "Bug-assist: Assisting fault localization in ANSI-C programs," in *Proceedings of Computer Aided Verification*, 2011, pp. 504–509.

[17] A. González, M. Cristiá, and C. Luna, "Verification of quantitative temporal properties in RealTime-DEVS," 2024, arXiv:2409.18732.

[18] G. Behrmann, A. David, and K. G. Larsen, "A tutorial on Uppaal," in *Formal Methods for the Design of Real-Time Systems*, ser. Lecture Notes in Computer Science, vol. 3185. Springer, 2004, pp. 200–236.

[19] K. G. Larsen, P. Pettersson, and W. Yi, "UPPAAL in a nutshell," *International Journal on Software Tools for Technology Transfer*, vol. 1, no. 1–2, pp. 134–152, 1997.

[20] P. Han, Z. Zhai, B. Nielsen, and U. Nyman, "A modeling framework for schedulability analysis of distributed avionics systems," 2018, arXiv:1803.11050.

[21] R. A. DeMillo, R. J. Lipton, and F. G. Sayward, "Hints on test data selection: Help for the practicing programmer," *Computer*, vol. 11, no. 4, pp. 34–41, 1978.

[22] R. Sebastiani and S. Tomasi, "Optimization modulo theories with linear rational costs," *ACM Transactions on Computational Logic*, vol. 16, no. 2, 2015.

[23] E. M. Clarke, O. Grumberg, S. Jha, Y. Lu, and H. Veith, "Counterexample-guided abstraction refinement," in *Proceedings of Computer Aided Verification*, 2000, pp. 154–169.

[24] Y. Jia and M. Harman, "An analysis and survey of the development of mutation testing," *IEEE Transactions on Software Engineering*, vol. 37, no. 5, pp. 649–678, 2011.

[25] L. de Moura and N. Bjørner, "Z3: An efficient SMT solver," in *Proceedings of Tools and Algorithms for the Construction and Analysis of Systems*, 2008, pp. 337–340.

[26] E. M. Clarke, O. Grumberg, and D. A. Peled, *Model Checking*. MIT Press, 1999.

[27] J. E. Hopcroft, R. Motwani, and J. D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 2nd ed. Addison-Wesley, 2001.